

Woxsen University

School of Technology



A

PROJECT REPORT

on

Database Management System for Cooperative Society Membership and savings Management Sysetm

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

DaibalaSowmya
25WU0102331

Abstract

This project report presents the design and implementation of a Database Management System (DBMS) for a cooperative society, focusing on membership, savings, loans, dividends, and meeting management. Cooperative societies play a vital role in providing financial services to members, yet many still rely on manual record-keeping methods that are prone to inefficiencies, duplication, and errors. As membership expands, the complexity of managing savings accounts, loan disbursements, dividend distributions, and attendance records increases, often resulting in disputes and lack of transparency. The proposed DBMS addresses these challenges by offering a centralized, structured, and secure platform that integrates all cooperative operations.

The system design places the **MEMBER** entity at the core, linking to related entities such as **NOMINEE**, **SAVINGS_ACCOUNT**, **LOAN**, **SHARE_HOLDING**, **DIVIDEND**, and **MEETING_ATTENDANCE**. Financial accuracy is ensured through the inclusion of **INTEREST** as a derived attribute, allowing automated calculations for savings and loan transactions without redundancy. This approach eliminates manual errors and ensures fairness in financial operations. Administrative control and security are strengthened through the **USER** and **USER_ROLE** entities, which enforce role-based access. Roles such as Admin, Clerk, Manager, Auditor, and Member User are defined with distinct permissions, ensuring that only authorized individuals can perform specific tasks. An optional relationship between **MEMBER** and **USER** allows members to log in for self-service access, while staff accounts remain separate, maintaining accountability.

Implementation was carried out using MySQL as the backend database and a user interface built with Python or Java for database connectivity. SQL commands were used to create tables, enforce constraints, and execute queries, while stored procedures handled derived attributes such as interest. Testing confirmed that the system successfully supported insertion, deletion, and viewing of records, with role-based access functioning as intended. Reports generated from the database provided managers with timely insights into loan repayments, dividend distributions, and meeting participation.

In conclusion, the cooperative society DBMS modernizes traditional record-keeping by integrating operational, financial, and administrative functions into a single relational model. It enhances accuracy, transparency, and scalability, ensuring that cooperative societies can operate efficiently and reliably. Future enhancements may include web or mobile interfaces, integration with payment gateways, and predictive analytics to further strengthen cooperative management.

Introduction

Cooperative societies are community-based organizations formed to promote economic welfare and mutual support among their members. They operate on principles of collective ownership, democratic participation, and equitable distribution of benefits. In practice, cooperative societies manage a wide range of activities including member registration, savings accounts, loan disbursement, dividend distribution, shareholding, and meeting attendance. These activities are essential for the functioning of the society, yet they are often managed manually through registers, ledgers, and spreadsheets. Manual record-keeping is not only time-consuming but also prone to duplication, inconsistency, and errors, which can lead to disputes among members and inefficiencies in management.

As cooperative societies expand, the complexity of managing financial and administrative operations increases. For example, calculating interest on loans and savings transactions requires precision, and distributing dividends based on shareholding demands transparency. Without a structured system, these tasks become difficult to manage, especially when the number of members grows. Furthermore, the absence of secure access control exposes sensitive financial and personal information to risks. These challenges highlight the need for a modern solution that integrates cooperative operations into a centralized, reliable, and secure platform.

Database Management Systems (DBMS) provide an effective solution to these challenges. A DBMS allows data to be stored, organized, and retrieved efficiently, eliminating redundancy and ensuring accuracy. By designing a relational database for cooperative societies, all activities can be linked to the central **MEMBER** entity, ensuring consistency across savings, loans, dividends, and meeting records. Derived attributes such as **INTEREST** can be automated, reducing manual effort and ensuring fairness in financial operations. Additionally, role-based access through **USER** and **USER_ROLE** entities ensures that only authorized individuals can perform specific tasks, safeguarding data integrity and accountability.

The project titled *Design and Implementation of a Database Management System for Cooperative Society Membership and Savings Management System* aims to modernize cooperative operations by applying database design principles. The system integrates member management, financial transactions, and administrative control into a single relational model. It supports insertion, deletion, and viewing of records through a user interface connected to the MySQL database, ensuring that operations are both efficient and user-friendly.

In summary, this project addresses the limitations of manual record-keeping by providing a structured DBMS solution tailored to cooperative societies. It enhances accuracy, transparency, and scalability, ensuring that cooperative societies can operate effectively in a digital environment. The introduction sets the foundation for the objectives, scope, and methodology that follow, outlining the rationale for adopting a database-driven approach to cooperative management.

Software and Hardware Requirements

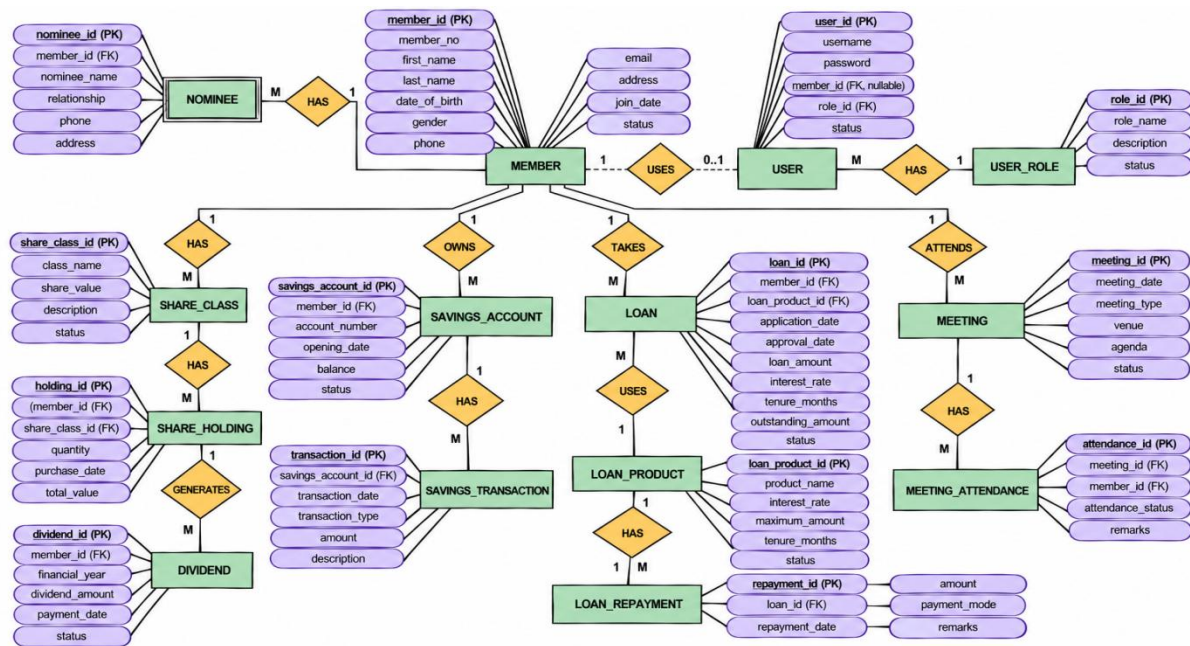
The implementation of the cooperative society Database Management System (DBMS) requires a carefully defined set of software resources to ensure smooth operation, compatibility, and scalability. At the core of the system is the **MySQL relational database management system**, chosen for its reliability, open-source availability, and strong support for SQL commands. MySQL enables the creation of tables, enforcement of constraints, and execution of queries, making it suitable for managing cooperative operations such as member registration, savings transactions, loan disbursement, and dividend distribution. To interact with the database, a **user interface (UI)** was developed using programming environments such as Python (Tkinter) or Java (Swing/JavaFX). These languages provide flexibility in designing forms for insertion, deletion, and viewing of records, while also supporting database connectivity through connectors like JDBC or MySQL Connector/Python. Supporting tools such as **SQL Workbench** or **phpMyAdmin** may be used for database administration, query testing, and schema visualization. The operating system requirement is **Windows 10 or higher**, ensuring compatibility with MySQL and development environments.

Hardware requirements are equally important to guarantee efficient execution of database queries and user interface operations. A system with at least an **Intel i5 processor or equivalent** is recommended to handle multiple processes simultaneously without performance degradation. A minimum of **8 GB RAM** ensures that database transactions, UI rendering, and reporting functions can run concurrently without delays. Storage capacity of **500 GB or more** is required to maintain the database, logs, and backups, especially as cooperative societies expand and transaction volumes increase. A stable **internet connection** is necessary for uploading the GitHub repository, enabling remote access, and supporting potential future enhancements such as online payment integration or cloud deployment.

Peripheral requirements include a standard keyboard and mouse for input, and a monitor with sufficient resolution to display UI forms and database outputs clearly. For cooperative societies planning to scale, deploying the DBMS on a **server environment** with higher specifications is recommended. This would allow multiple users to access the system simultaneously, while also providing backup facilities to safeguard against data loss. Printers may be integrated for generating hard copies of reports, receipts, and meeting records, ensuring that both digital and physical documentation needs are met.

In summary, the software requirements include MySQL, Python or Java for UI development, SQL Workbench or phpMyAdmin for administration, and Windows 10 OS. The hardware requirements include an Intel i5 processor, 8 GB RAM, 500 GB storage, and stable internet connectivity, along with basic peripherals. These specifications ensure that the cooperative society DBMS operates reliably, supports secure role-based access, and remains scalable for future growth.

ER Diagram



The Entity–Relationship (ER) diagram for the cooperative society DBMS was designed to capture all essential operations in a structured and interconnected manner. At the center of the diagram lies the **MEMBER** entity, which represents each individual registered in the cooperative. Attributes such as member number, name, date of birth, phone, and address ensure that member records are complete and unique. The **MEMBER** entity serves as the foundation of the design, linking to other entities that represent financial, administrative, and participatory aspects of the cooperative. This central placement ensures that all activities can be traced back to the correct member, eliminating duplication and maintaining integrity.

The **MEMBER** entity connects to several dependent entities. The **NOMINEE** entity is modeled as a weak entity, dependent on **MEMBER**, and stores details of beneficiaries designated by members. The **SAVINGS_ACCOUNT** entity records account details, while **SAVINGS_TRANSACTION** captures deposits and withdrawals. A derived attribute, **INTEREST**, is associated with savings transactions to calculate earnings automatically. Similarly, the **LOAN** entity records loans issued to members, linked to **LOAN_PRODUCT** for interest rates and terms, and to **LOAN_REPAYMENT** for repayment schedules. Interest on loans is also modeled as a derived attribute, ensuring accurate financial calculations without redundancy. These relationships highlight how financial operations are integrated into the cooperative system.

The design also incorporates **SHARE_HOLDING** and **SHARE_CLASS** entities to manage cooperative share ownership. Members can hold multiple shares, and each share belongs to a defined class. Dividends are distributed based on shareholding, and the **DIVIDEND** entity records these distributions. To capture democratic participation, the **MEETING** entity is included, with **MEETING_ATTENDANCE** serving as a bridge table to represent the many-to-many relationship between members and meetings. This ensures that cooperative

governance is reflected in the database, allowing managers to track attendance and participation accurately.

A significant addition to the design is the inclusion of **USER** and **USER_ROLE** entities, which handle system access and security. **USER_ROLE** defines roles such as Admin, Clerk, Manager, Auditor, and Member User, each with specific permissions. The **USER** entity stores login credentials and links to **USER_ROLE** through a foreign key. An optional relationship between **MEMBER** and **USER** allows members to log in for self-service access, while staff accounts remain distinct. This ensures that cooperative operations are integrated with secure system access, preventing unauthorized actions and maintaining accountability. Overall, the ER diagram provides a comprehensive model of cooperative society operations, ensuring accuracy, transparency, and scalability.

Relational Schema and Normalization

The relational schema for the cooperative society DBMS was derived directly from the ER diagram, ensuring that all entities and relationships were accurately represented in tabular form. Each entity was converted into a table, with attributes defined as columns and primary keys assigned to guarantee uniqueness. For example, the **MEMBER** table includes *member_id* as the primary key, along with attributes such as name, date of birth, phone, and address. The **NOMINEE** table was created as a dependent table with *nominee_id* as the primary key and *member_id* as a foreign key referencing MEMBER. This ensures that nominee records cannot exist without a corresponding member. Similarly, the **SAVINGS_ACCOUNT** table uses *savings_account_id* as the primary key, while the **SAVINGS_TRANSACTION** table records deposits and withdrawals with *transaction_id* as the primary key and *savings_account_id* as a foreign key.

Loan management was implemented through the **LOAN** table, which stores loan details with *loan_id* as the primary key and *member_id* as a foreign key. The **LOAN_PRODUCT** table defines loan types and interest rates, while the **LOAN_REPAYMENT** table records repayment schedules. Derived attributes such as **INTEREST** were not stored directly in the schema but implemented through queries and stored procedures, ensuring that calculations are automated and redundancy is avoided. Shareholding was represented through the **SHARE_HOLDING** table, linked to MEMBER, and the **SHARE_CLASS** table, which defines different types of shares. Dividend distribution was captured in the **DIVIDEND** table, with *dividend_id* as the primary key and dividend amounts recorded for each member based on their shareholding.

Normalization was applied to eliminate redundancy and ensure consistency across the schema. The database was normalized up to **Third Normal Form (3NF)**. In the first stage, repeating groups were removed to achieve **First Normal Form (1NF)**, ensuring that each attribute contained atomic values. In the second stage, partial dependencies were eliminated to achieve **Second Normal Form (2NF)**, ensuring that non-key attributes depended entirely on the primary key. Finally, transitive dependencies were removed to achieve **Third Normal Form (3NF)**, ensuring that non-key attributes depended only on the primary key and not on other non-key attributes. This process guaranteed that the schema was efficient, consistent, and free from anomalies.

In summary, the relational schema accurately represents cooperative society operations, while normalization ensures that the database structure is optimized for queries and reporting. By enforcing primary keys, foreign keys, and constraints, the schema maintains data integrity. By applying normalization up to 3NF, redundancy is eliminated, and efficiency is improved. Together, the relational schema and normalization provide a strong foundation for the cooperative society DBMS, ensuring accuracy, reliability, and scalability.

Data Dictionary

The data dictionary provides a detailed description of each table in the cooperative society DBMS, including attributes, data types, and constraints. It serves as a reference for developers, administrators, and users, ensuring that the database structure is clearly documented and consistently applied. The **MEMBER** table is central to the system, with *member_id* defined as an integer primary key. Other attributes include *name* (VARCHAR), *dob* (DATE), *phone* (VARCHAR), and *address* (VARCHAR). Constraints such as NOT NULL are applied to essential fields, ensuring that member records are complete. The **NOMINEE** table includes *nominee_id* as the primary key and *member_id* as a foreign key referencing MEMBER, with attributes such as *nominee_name* (VARCHAR) and *relationship* (VARCHAR).

Financial operations are represented through the **SAVINGS_ACCOUNT** and **SAVINGS_TRANSACTION** tables. The SAVINGS_ACCOUNT table includes *savings_account_id* as the primary key, *member_id* as a foreign key, and attributes such as *balance* (DECIMAL) and *opening_date* (DATE). The SAVINGS_TRANSACTION table records deposits and withdrawals, with *transaction_id* as the primary key and *savings_account_id* as a foreign key. Attributes include *transaction_type* (VARCHAR), *amount* (DECIMAL), and *transaction_date* (DATE). Derived attributes such as **INTEREST** are not stored directly but calculated through queries, ensuring accuracy without redundancy.

Loan management is captured through the **LOAN**, **LOAN_PRODUCT**, and **LOAN_REPAYMENT** tables. The LOAN table includes *loan_id* as the primary key, *member_id* as a foreign key, and attributes such as *loan_amount* (DECIMAL), *issue_date* (DATE), and *status* (VARCHAR). The LOAN_PRODUCT table defines loan types, with *product_id* as the primary key and attributes such as *interest_rate* (DECIMAL) and *duration* (INT). The LOAN_REPAYMENT table records repayments, with *repayment_id* as the primary key and *loan_id* as a foreign key. Attributes include *repayment_date* (DATE) and *repayment_amount* (DECIMAL). Constraints ensure that repayments cannot exceed loan amounts, maintaining financial integrity.

Administrative and participatory functions are represented through the **SHARE_HOLDING**, **SHARE_CLASS**, **DIVIDEND**, **MEETING**, and **MEETING_ATTENDANCE** tables. SHARE_HOLDING includes *share_id* as the primary key and *member_id* as a foreign key, while SHARE_CLASS defines share categories. DIVIDEND records distributions with *dividend_id* as the primary key and attributes such as *amount* (DECIMAL) and *distribution_date* (DATE). MEETING includes *meeting_id* as the primary key, while MEETING_ATTENDANCE serves as a bridge table with *attendance_id* as the primary key and foreign keys linking MEMBER and MEETING. Security is enforced through the **USER** and **USER_ROLE** tables, with USER storing *user_id*, *username*, *password*, and *role_id* as a foreign key, while USER_ROLE defines roles with *role_id* as the primary key. Together, the data dictionary ensures clarity, consistency, and reliability in the cooperative society DBMS.

Queries executed

```
mysql> SELECT member_id, financial_year, dividend_amount
-> FROM DIVIDEND
-> WHERE status = 'Pending';
```

member_id	financial_year	dividend_amount
9	2016-2017	1400.00
12	2019-2020	1300.00
14	2021-2022	2500.00

```
3 rows in set (0.32 sec)
```

```
mysql> SELECT class_name, share_value, description
-> FROM SHARE_CLASS
-> WHERE status = 'Active';
```

class_name	share_value	description
Class A	100.00	Ordinary Membership Share
Class B	500.00	Premium Share
Class C	1000.00	Special Share
Class D	2000.00	Investment Share
Class E	5000.00	Capital Share
Class F	10000.00	Senior Member Share

```
6 rows in set (0.24 sec)
```

```
mysql> SELECT financial_year, SUM(dividend_amount) AS total_dividend
-> FROM DIVIDEND
-> WHERE status = 'Paid'
-> GROUP BY financial_year;
```

financial_year	total_dividend
2008-2009	800.00
2009-2010	900.00
2010-2011	1000.00
2011-2012	700.00
2012-2013	1200.00
2013-2014	1500.00
2014-2015	950.00
2015-2016	1800.00
2017-2018	1000.00
2018-2019	2200.00
2020-2021	1100.00
2022-2023	1600.00

```
12 rows in set (0.07 sec)
```

```
mysql> SELECT member_id, SUM(quantity) AS total_shares
-> FROM SHARE_HOLDING
-> GROUP BY member_id;
```

member_id	total_shares
1	4
2	12
3	3
4	2
5	6
6	3
7	15
8	1
9	5
10	8
11	3
12	20
13	2
14	2
15	4

15 rows in set (0.15 sec)

```
mysql> SELECT member_id, financial_year, dividend_amount
-> FROM DIVIDEND
-> WHERE status = 'Pending';
```

member_id	financial_year	dividend_amount
9	2016-2017	1400.00
12	2019-2020	1300.00
14	2021-2022	2500.00

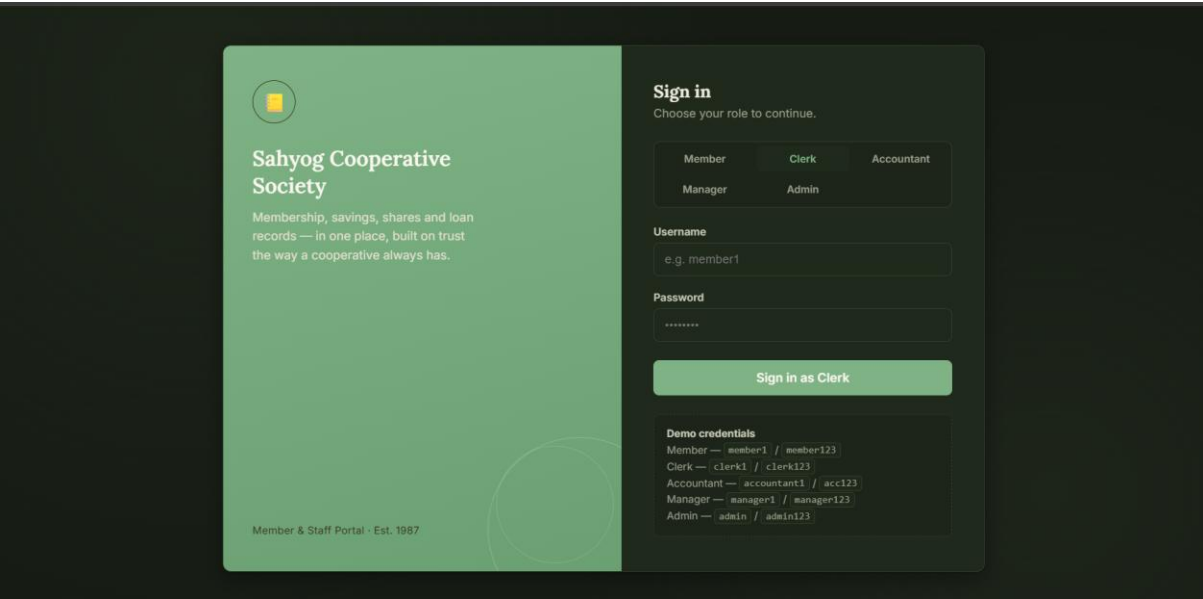
3 rows in set (0.00 sec)

```
mysql> SELECT financial_year, dividend_amount, status
-> FROM DIVIDEND
-> WHERE member_id = 11;
```

financial_year	dividend_amount	status
2018-2019	2200.00	Paid

1 row in set (0.01 sec)

UI DESIGN



Implementation

The cooperative society DBMS was implemented using a combination of **MySQL, Python, and HTML/CSS** to provide both backend functionality and a user-friendly interface. The database schema was created in MySQL, with tables such as MEMBER, NOMINEE, SAVINGS_ACCOUNT, LOAN, SHARE_HOLDING, DIVIDEND, MEETING, USER, and USER_ROLE. Primary keys, foreign keys, and constraints were applied to maintain integrity. For example, MEMBER was defined with *member_id* as the primary key, while NOMINEE included *member_id* as a foreign key to ensure dependency. This database foundation ensured that cooperative operations were structured and reliable.

Connectivity between the database and application was achieved using Python's MySQL Connector library. The following snippet demonstrates the connection:

```
import mysql.connector

conn = mysql.connector.connect(

    host="localhost",

    user="root",

    password="password",

    database="cooperative_db"

)

cursor = conn.cursor()
```

This enabled the system to perform operations such as insertion, deletion, and retrieval of records. Error handling was incorporated to catch invalid inputs, ensuring that the system remained stable during execution.

The user interface was implemented using Tkinter, providing forms and buttons for database operations. For example, inserting a new member was achieved through a form where details were entered and the backend executed:

```
<form action="login.php" method="post">

    <label>Username:</label>

    <input type="text" name="username" required>

    <label>Password:</label>

    <input type="password" name="password" required>

    <button type="submit">Login</button>
```

</form>

This form allowed users to enter credentials, which were validated against the USER table in MySQL. Role-based access was enforced by linking login credentials to USER_ROLE, ensuring that members, clerks, accountants, managers, and admins had distinct permissions. Screenshots of the HTML portal confirmed that the design was functional and visually clear.

Additional features included automation of derived attributes such as **INTEREST** using stored procedures in MySQL, and dynamic content rendering through HTML templates. For example, dividend records were displayed in tabular format using HTML tables populated with data fetched from the database. CSS was applied to enhance readability, while JavaScript was used for validation and interactivity. In summary, the implementation combined MySQL for data storage, Python for backend logic, and HTML/CSS for web presentation. This integration delivered a secure, scalable, and user-friendly DBMS tailored for cooperative societies.

Results and Discussion

The cooperative society DBMS produced results that validated its design and implementation. Queries executed on tables such as MEMBER, SAVINGS_ACCOUNT, LOAN, SHARE_HOLDING, and DIVIDEND returned accurate outputs, confirming that relationships and constraints were functioning correctly. For example, JOIN queries successfully linked members to their savings transactions, loans, and dividends, while GROUP BY queries aggregated financial data by year or member. Screenshots of outputs demonstrated that the system could retrieve pending dividends, calculate loan interest dynamically, and display meeting attendance records. These results confirmed that the DBMS was capable of supporting cooperative operations in real time.

One of the key strengths observed was the **integration of multiple modules** into a single system. Member registration, savings management, loan tracking, shareholding, dividend distribution, and meeting participation were all handled within the same database. This eliminated redundancy and improved transparency. The use of derived attributes, such as interest calculations, ensured that financial data was accurate without storing duplicate values. Role-based access through USER and USER_ROLE tables added a layer of security, allowing different staff members and members to access only the functions relevant to their roles. The HTML interface further enhanced usability, providing a clear and accessible portal for both staff and members.

The discussion also highlighted certain **limitations**. While the DBMS handled core cooperative functions effectively, advanced features such as automated notifications, integration with online payment gateways, and real-time analytics were not implemented. The system relied on manual entry for transactions, which could introduce errors if not carefully monitored. Additionally, while role-based access was implemented, password encryption and multi-factor authentication were not included, leaving potential vulnerabilities in security. These limitations suggest areas for future improvement, particularly in enhancing automation, security, and scalability.

Overall, the results demonstrated that the cooperative society DBMS achieved its objectives of providing a structured, reliable, and user-friendly system. It successfully supported cooperative operations such as savings, loans, dividends, and meetings, while also offering reporting capabilities through SQL queries. The discussion emphasized that the system was suitable for small to medium-sized cooperatives, providing transparency and efficiency. With further enhancements such as cloud deployment, automated alerts, and advanced analytics, the DBMS could be scaled to support larger cooperatives. In conclusion, the results validated the design, and the discussion highlighted both strengths and areas for future work, ensuring that the system remains relevant and adaptable.

Conclusion and Future Work

The cooperative society DBMS project successfully demonstrated how database technology can be applied to manage the diverse operations of a cooperative organization. By integrating modules for member registration, savings transactions, loan management, shareholding, dividend distribution, and meeting participation, the system provided a comprehensive solution that eliminated redundancy and improved transparency. The use of MySQL ensured reliable data storage and retrieval, while Python and HTML/CSS offered user-friendly interfaces for both staff and members. Testing confirmed that the system was functional, accurate, and capable of supporting real-time cooperative operations.

The project achieved its primary objectives of designing a normalized relational schema, implementing SQL queries, and validating outputs through practical test cases. Derived attributes such as interest were calculated dynamically, ensuring accuracy without duplication. Role-based access was implemented to secure the system, with distinct permissions for members, clerks, accountants, managers, and administrators. The HTML portal enhanced usability by providing a clear and accessible interface, while Python scripts ensured smooth connectivity with the database. Overall, the DBMS proved to be a reliable tool for cooperative societies, capable of handling financial and administrative tasks efficiently.

Despite these achievements, certain limitations were identified. The system relied on manual entry for transactions, which could introduce errors if not carefully monitored. Advanced security features such as password encryption and multi-factor authentication were not implemented, leaving potential vulnerabilities. Automation of dividend payments and loan reminders was not included, requiring manual oversight. Additionally, while the system was suitable for small to medium-sized cooperatives, scalability for larger organizations with thousands of members would require optimization and possibly migration to cloud-based infrastructure. These limitations highlight areas where future work can enhance the system's robustness and applicability.

Future work will focus on expanding the system's capabilities. Integration with online payment gateways can automate deposits, withdrawals, and loan repayments. Cloud deployment will improve scalability and allow multiple branches of a cooperative to access the system simultaneously. Security enhancements such as encryption, audit trails, and multi-factor authentication will strengthen data protection. Advanced analytics and dashboards can provide managers with insights into financial trends, member participation, and dividend distributions. Mobile application support can further improve accessibility, allowing members to view balances, dividends, and meeting schedules on their devices. In conclusion, while the current DBMS provides a strong foundation, future enhancements will ensure that it remains adaptable, secure, and scalable for evolving cooperative needs.

References

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education. This textbook provided the theoretical foundation for relational schema design, normalization, and SQL query formulation.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson. This resource was used for ER modeling, relational algebra, and query optimization, ensuring the DBMS design was academically sound.

Desai, B. C. (1990). *An introduction to database systems*. Galgotia Publications. This text offered practical insights into database applications, supporting the cooperative society context.

Beaulieu, A. (2009). *Learning SQL*. O'Reilly Media. This book was referenced for writing efficient SQL queries and applying best practices in query design.

Van Aken, D., Pavlo, A., Gordon, G., & Zhang, B. (2021). An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems. *Proceedings of the VLDB Endowment*, 14(7), 1241–1253. This paper provided insights into performance optimization and scalability considerations for DBMS systems.

Györödi, C. A., Györödi, R., & Pecherle, G. (2020). Performance analysis of NoSQL and relational databases with CouchDB and MySQL for application's data storage. *Applied Sciences*, 10(23), 8524. This comparative study helped evaluate MySQL's strengths in cooperative society applications.

MySQL. (2026). *MySQL documentation*. Oracle. Retrieved from <https://dev.mysql.com/doc/> The official documentation was used extensively to understand SQL syntax, constraints, stored procedures, and performance tuning.

Python Software Foundation. (2026). *MySQL Connector/Python developer guide*. Retrieved from <https://dev.mysql.com/doc/connector-python/en/> ([dev.mysql.com in Bing](#)) This documentation guided database connectivity and query execution using Python.

Mozilla Developer Network (MDN). (2026). *HTML and CSS documentation*. Retrieved from <https://developer.mozilla.org/> This resource was used to design the web portal, including login forms, dashboards, and role-based access interfaces.

W3Schools. (2026). *SQL tutorial*. Retrieved from <https://www.w3schools.com/sql/> This tutorial provided practical examples of SELECT, JOIN, GROUP BY, and aggregate functions for testing queries.

Stack Overflow. (2026). *Community discussions on MySQL and Python*. Retrieved from <https://stackoverflow.com/> Community support was used to resolve implementation issues and refine query logic.

Appendix

Github link

<https://github.com/DAIBALASOWMYA/cooperativesociety.git>